

week6_Visualizing with ggplot2

Macarena López-Mayorga

2026-05-05

First, we have to load dplyr

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

This message means that R is masking (or hiding) the base R stats and base package functions with identical names to prioritize the functions of dplyr when code is run.

Now, we are going to work with dataset from week04

```
StorageLocation <- file.path("C:/Users/Macarena/CytometryInR/course/04_IntroToTidyverse/data")
StorageLocation
```

```
TheCSV <- list.files(StorageLocation, full.names=TRUE)
TheCSV
```

```
Data <- read.csv(TheCSV, check.names=FALSE)
head(Data, 3)
```

bid timepoint Condition

```

1 INF0052 0 Ctrl 2 INF0100 0 Ctrl 3 INF0100 4 Ctrl Date infant_sex ptype root 1 2025-07-26
Male HEU-hi 2098368 2 2025-07-26 Male HEU-lo 2020184 3 2025-07-26 Male HEU-lo 1155040
singletsFSC singletsSSC singletsSSCB 1 1894070 1666179 1537396 2 1791890 1697083
1579098 3 1033320 875465 845446 CD45 NotMonocytes nonDebris 1 0.5952943 0.8820349
0.8627649 2 0.9106762 0.9052256 0.8602660 3 0.9705765 0.9845400 0.9578793 lymphocytes
live Dump+ 1 0.6420138 0.9020581 0.21090996 2 0.2145848 0.8908981 0.06252775 3 0.7403110
0.8757665 0.20023803 Dump- Tcells Vd2+ 1 0.6911482 0.2804264 0.008120361 2 0.8283703
0.6748298 0.007265620 3 0.6755285 0.6119129 0.004651313 Vd2- Va7.2+ Va7.2- 1 0.9918796
0.01448070 0.9773989 2 0.9927344 0.01577499 0.9769594 3 0.9953487 0.01579402 0.9795547
CD4+ CD4- CD8+ 1 0.6341164 0.3432825 0.2734826 2 0.6119112 0.3650482 0.3357696 3
0.6639621 0.3155925 0.2862104 CD8- Tcells_count 1 0.06979990 164771 2 0.02927858 208241 3
0.02938209 371723 lymphocytes_count Monocytes 1 587573 0.11796509 2 308583 0.09477437
3 607477 0.01545999 Debris CD45_count 1 0.13723513 915203 2 0.13973396 1438047 3
0.04212072 820570

```

And now we are going to determine what type of values contains each column:

```
str(Data)
```

```

'data.frame': 196 obs. of 31 variables: $ bid : chr "INF0052" "INF0100" "INF0100" "INF0100"
... $ timepoint : int 0 0 4 9 0 4 9 0 4 9 0 ... $ Condition : chr "Ctrl" "Ctrl" "Ctrl" "Ctrl" ... $ Date :
chr "2025-07-26" "2025-07-26" "2025-07-26" "2025-07-26" ... $ infant_sex : chr "Male" "Male"
"Male" "Male" ... $ ptype : chr "HEU-hi" "HEU-lo" "HEU-lo" "HEU-lo" ... $ root : int 2098368
2020184 1155040 358624 1362216 1044808 1434840 972056 1521928 2363512 ... $ singletsFSC
: int 1894070 1791890 1033320 328624 1206309 917398 1265022 875707 1359574 2136616 ...
$ singletsSSC : int 1666179 1697083 875465 289327 1032946 735579 988445 767323 1175755
1875394 ... $ singletsSSCB : int 1537396 1579098 845446 276289 982736 685592 940454 718000
1097478 1732620 ... $ CD45 : num 0.595 0.911 0.971 0.982 0.957 ... $ NotMonocytes : num 0.882
0.905 0.985 0.986 0.956 ... $ nonDebris : num 0.863 0.86 0.958 0.941 0.841 ... $ lymphocytes :
num 0.642 0.215 0.74 0.651 0.705 ... $ live : num 0.902 0.891 0.876 0.915 0.895 ... $ Dump+
: num 0.2109 0.0625 0.2002 0.2147 0.3383 ... $ Dump- : num 0.691 0.828 0.676 0.701 0.557 ...
$ Tcells : num 0.28 0.675 0.612 0.631 0.44 ... $ Vd2+ : num 0.00812 0.00727 0.00465 0.01135
0.00475 ... $ Vd2- : num 0.992 0.993 0.995 0.989 0.995 ... $ Va7.2+ : num 0.0145 0.0158 0.0158
0.017 0.0133 ... $ Va7.2- : num 0.977 0.977 0.98 0.972 0.982 ... $ CD4+ : num 0.634 0.612 0.664
0.438 0.739 ... $ CD4- : num 0.343 0.365 0.316 0.534 0.243 ... $ CD8+ : num 0.273 0.336 0.286
0.486 0.195 ... $ CD8- : num 0.0698 0.0293 0.0294 0.0476 0.0476 ... $ Tcells_count : int 164771
208241 371723 111552 291777 271870 487937 220634 415867 184930 ... $ lymphocytes_count:
int 587573 308583 607477 176662 663667 510730 726238 451047 710964 652155 ... $ Monocytes
: num 0.118 0.0948 0.0155 0.0145 0.0444 ... $ Debris : num 0.1372 0.1397 0.0421 0.0587 0.1592
... $ CD45_count : int 915203 1438047 820570 271304 940733 675857 921660 701657 1066884
1017713 ... The type of values that a column contains are particularly important to be aware
of when working with ggplot2 plots. For example, we can see that several columns contain
character values:

```

```
::: {.cell}
```

```
Data |> select(bid, Condition, Date, infant_sex, ptype) |> head(6) |> str()
```

```
:::
```

```
‘data.frame’: 6 obs. of 5 variables: $ bid : chr “INF0052” “INF0100” “INF0100” “INF0100”  
... $ Condition : chr “Ctrl” “Ctrl” “Ctrl” “Ctrl” ... $ Date : chr “2025-07-26” “2025-07-26”  
“2025-07-26” “2025-07-26” ... $ infant_sex: chr “Male” “Male” “Male” “Male” ... $ ptype : chr  
“HEU-hi” “HEU-lo” “HEU-lo” “HEU-lo” ...
```

This is other way to do same but using tidyselect package (dplyr):

```
::: {.cell}
```

```
Data |> select(where(is.character)) |> head(6) |> str()
```

```
:::
```

```
‘data.frame’: 6 obs. of 5 variables: $ bid : chr “INF0052” “INF0100” “INF0100” “INF0100”  
... $ Condition : chr “Ctrl” “Ctrl” “Ctrl” “Ctrl” ... $ Date : chr “2025-07-26” “2025-07-26”  
“2025-07-26” “2025-07-26” ... $ infant_sex: chr “Male” “Male” “Male” “Male” ... $ ptype : chr  
“HEU-hi” “HEU-lo” “HEU-lo” “HEU-lo” ... > Data |> select(where(is.character)) |> head(6)  
|> str() ‘data.frame’: 6 obs. of 5 variables: $ bid : chr “INF0052” “INF0100” “INF0100”  
“INF0100” ... $ Condition : chr “Ctrl” “Ctrl” “Ctrl” “Ctrl” ... $ Date : chr “2025-07-26”  
“2025-07-26” “2025-07-26” “2025-07-26” ... $ infant_sex: chr “Male” “Male” “Male” “Male” ...  
$ ptype : chr “HEU-hi” “HEU-lo” “HEU-lo” “HEU-lo” ...
```

We can also ask for numeric values or logical values:

```
::: {.cell}
```

```
Data |> select(where(is.numeric)) |> head(6) |> str()
```

```
:::
```

```
‘data.frame’: 6 obs. of 26 variables: $ timepoint : int 0 0 4 9 0 4 $ root : int 2098368 2020184  
1155040 358624 1362216 1044808 $ singletsFSC : int 1894070 1791890 1033320 328624 1206309  
917398 $ singletsSSC : int 1666179 1697083 875465 289327 1032946 735579 $ singletsSSCB  
: int 1537396 1579098 845446 276289 982736 685592 $ CD45 : num 0.595 0.911 0.971 0.982  
0.957 ... $ NotMonocytes : num 0.882 0.905 0.985 0.986 0.956 ... $ nonDebris : num 0.863 0.86  
0.958 0.941 0.841 ... $ lymphocytes : num 0.642 0.215 0.74 0.651 0.705 ... $ live : num 0.902  
0.891 0.876 0.915 0.895 ... $ Dump+ : num 0.2109 0.0625 0.2002 0.2147 0.3383 ... $ Dump- :  
num 0.691 0.828 0.676 0.701 0.557 ... $ Tcells : num 0.28 0.675 0.612 0.631 0.44 ... $ Vd2+ :  
num 0.00812 0.00727 0.00465 0.01135 0.00475 ... $ Vd2- : num 0.992 0.993 0.995 0.989 0.995
```

```
... $ Va7.2+ : num 0.0145 0.0158 0.0158 0.017 0.0133 ... $ Va7.2- : num 0.977 0.977 0.98 0.972
0.982 ... $ CD4+ : num 0.634 0.612 0.664 0.438 0.739 ... $ CD4- : num 0.343 0.365 0.316
0.534 0.243 ... $ CD8+ : num 0.273 0.336 0.286 0.486 0.195 ... $ CD8- : num 0.0698 0.0293
0.0294 0.0476 0.0476 ... $ Tcells_count : int 164771 208241 371723 111552 291777 271870 $
lymphocytes_count: int 587573 308583 607477 176662 663667 510730 $ Monocytes : num
0.118 0.0948 0.0155 0.0145 0.0444 ... $ Debris : num 0.1372 0.1397 0.0421 0.0587 0.1592 ... $
CD45_count : int 915203 1438047 820570 271304 940733 675857

::: {.cell}
```

```
Data |> select(where(is.logical)) |> head(6) |> str()
```

```
:::
```

```
‘data.frame’: 6 obs. of 0 variables
```

```
# Reformatting Column Value Types Date column is currently consider as a character value:
```

```
::: {.cell}
```

```
Data |> select(Date) |> str()
```

```
:::
```

```
‘data.frame’: 196 obs. of 1 variable: $ Date: chr “2025-07-26” “2025-07-26” “2025-07-26”
“2025-07-26” ...
```

Character values when plotting are generally treated as categorical factors. When plotted, these will arrange according to alphabetical order. In this particular case, this may not be an issue:

```
::: {.cell}
```

```
Data |> pull(Date) |> sort() |> unique()
```

```
:::
```

```
[1] “2025-07-26” “2025-07-29” “2025-07-31” “2025-08-05” [5] “2025-08-07” “2025-08-22” “2025-
08-28” “2025-08-30”
```

However, if year/month/day is formatted differently, as character values, the alphabetical reordering could result in your dates being scrambled:

```
AlternateFormat <- c("26-07-2025", "27-08-2019", "09-09-2025", "16-03-2026")
AlternateFormat |> sort() |> unique()
```

```
[1] "09-09-2025" "16-03-2026" "26-07-2025" "27-08-2019"
```

Because of this, I generally recommend reformatting these character type values over to Date type values. I will generally use the tidyverse lubridate package, as it has various functions that can handle the date format variation. In this case, since the character values appear year-month-day, we can use the ymd() function to update the column type:

```
library(lubridate)
```

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

```
date, intersect, setdiff, union
```

```
Data$Date <- ymd(Data$Date)
str(Data[,1:5])
```

```
'data.frame': 196 obs. of 5 variables: $ bid : chr "INF0052" "INF0100" "INF0100" "INF0100"
... $ timepoint : int 0 0 4 9 0 4 9 0 4 9 0 ... $ Condition : chr "Ctrl" "Ctrl" "Ctrl" "Ctrl" ... $
Date : Date, format: "2025-07-26" ... $ infant_sex: chr "Male" "Male" "Male" "Male" ...
```

You can notice, str() now returns the column type as "Date".

Factors

For columns with numeric type values, we previously discussed the difference between integer (ie. whole) and double (containing decimal point) values. When it comes to plotting using ggplot, numeric values will generally be treated as continuous values unless we specify they should be treated as categorical values for plotting.

For example, if we look at the Timepoint column, we can see its numeric values are discrete, corresponding to the timepoint in months when the blood sample was collected (0, 4, 9):

```
Data |> pull(timepoint) |> unique()
```

```
[1] 0 4 9
```

If we want to avoid having ggplot treating these values as continuous, and instead see them as categorical, we will need to specify to R to treat these as such. We can do this by converting them into factors, using the factor() function:

```
Data1 <- Data
Data1$timepoint <- factor(Data1$timepoint)
str(Data1[,1:3])
```

```
‘data.frame’: 196 obs. of 3 variables: $ bid : chr “INF0052” “INF0100” “INF0100” “INF0100”  
... $ timepoint: Factor w/ 3 levels “0”,“4”,“9”: 1 1 2 3 1 2 3 2 3 1 ... $ Condition: chr “Ctrl”  
“Ctrl” “Ctrl” “Ctrl” ...
```

Let’s try creating a boxplot, with the different timepoints on the x-axis, and the frequency of T cells in the CD45+ gate on the y-axis. Let’s go ahead and actually convert timepoint over to a categorical factor for this example, and mutate the frequency column:

```
::: {.cell}
```

```
Data <- Data |>  
  mutate(TcellProportion=Tcells_count/CD45_count) |>  
  mutate(TcellFrequency=TcellProportion *100) |>  
  mutate(TcellFrequency=round(TcellFrequency, 1))  
  
Data$timepoint <- factor(Data$timepoint)
```

```
:::
```

```
::: {.cell}
```

```
library(ggplot2)
```

```
:::
```

```
::: {.cell}
```

```
Plot <- ggplot(Data)  
Plot
```

```
:::
```

(the result is a grey screen)

As you can see, nothing is yet outputted at this stage. Let’s however peak behind the curtain.

```
::: {.cell}
```

```
str(Plot)
```

```
:::
```

```
@ data :‘data.frame’: 196 obs. of 33 variables: .. $ bid : chr “INF0052” “INF0100” “INF0100”  
“INF0100” ... .. $ timepoint : Factor w/ 3 levels “0”,“4”,“9”: 1 1 2 3 1 2 3 2 3 1 ... .. $  
Condition : chr “Ctrl” “Ctrl” “Ctrl” “Ctrl” ... .. $ Date : Date, format: “2025-07-26” ... ..
```

```

$ infant_sex : chr "Male" "Male" "Male" "Male" ... .. $ ptype : chr "HEU-hi" "HEU-lo"
"HEU-lo" "HEU-lo" ... .. $ root : int 2098368 2020184 1155040 358624 1362216 1044808
1434840 972056 1521928 2363512 ... .. $ singletsFSC : int 1894070 1791890 1033320 328624
1206309 917398 1265022 875707 1359574 2136616 ... .. $ singletsSSC : int 1666179 1697083
875465 289327 1032946 735579 988445 767323 1175755 1875394 ... .. $ singletsSSCB : int
1537396 1579098 845446 276289 982736 685592 940454 718000 1097478 1732620 ... .. $ CD45
: num 0.595 0.911 0.971 0.982 0.957 ... .. $ NotMonocytes : num 0.882 0.905 0.985 0.986
0.956 ... .. $ nonDebris : num 0.863 0.86 0.958 0.941 0.841 ... .. $ lymphocytes : num 0.642
0.215 0.74 0.651 0.705 ... .. $ live : num 0.902 0.891 0.876 0.915 0.895 ... .. $ Dump+ : num
0.2109 0.0625 0.2002 0.2147 0.3383 ... .. $ Dump- : num 0.691 0.828 0.676 0.701 0.557 ...
.. $ Tcells : num 0.28 0.675 0.612 0.631 0.44 ... .. $ Vd2+ : num 0.00812 0.00727 0.00465
0.01135 0.00475 ... .. $ Vd2- : num 0.992 0.993 0.995 0.989 0.995 ... .. $ Va7.2+ : num
0.0145 0.0158 0.0158 0.017 0.0133 ... .. $ Va7.2- : num 0.977 0.977 0.98 0.972 0.982 ... ..
$ CD4+ : num 0.634 0.612 0.664 0.438 0.739 ... .. $ CD4- : num 0.343 0.365 0.316 0.534
0.243 ... .. $ CD8+ : num 0.273 0.336 0.286 0.486 0.195 ... .. $ CD8- : num 0.0698 0.0293
0.0294 0.0476 0.0476 ... .. $ Tcells_count : int 164771 208241 371723 111552 291777 271870
487937 220634 415867 184930 ... .. $ lymphocytes_count: int 587573 308583 607477 176662
663667 510730 726238 451047 710964 652155 ... .. $ Monocytes : num 0.118 0.0948 0.0155
0.0145 0.0444 ... .. $ Debris : num 0.1372 0.1397 0.0421 0.0587 0.1592 ... .. $ CD45_count
: int 915203 1438047 820570 271304 940733 675857 921660 701657 1066884 1017713 ... .. $
TcellProportion : num 0.18 0.145 0.453 0.411 0.31 ... .. $ TcellFrequency : num 18 14.5 45.3
41.1 31 40.2 52.9 31.4 39 18.2 ... @ layers : list() @ scales :Classes 'ScalesList', 'ggproto', 'gg'
<ggproto object: Class ScalesList, gg> add: function add_defaults: function add_missing:
function backtransform_df: function clone: function find: function get_scales: function
has_scale: function input: function map_df: function n: function non_position_scales:
function scales: NULL set_palettes: function train_df: function transform_df: function
super: <ggproto object: Class ScalesList, gg> @ guides :Classes 'Guides', 'ggproto', 'gg'
<ggproto object: Class Guides, gg> add: function assemble: function build: function draw:
function get_custom: function get_guide: function get_params: function get_position:
function guides: NULL merge: function missing: <ggproto object: Class GuideNone,
Guide, gg> add_title: function arrange_layout: function assemble_drawing: function
available_aes: any build_decor: function build_labels: function build_ticks: function
build_title: function draw: function draw_early_exit: function elements: list extract_decor:
function extract_key: function extract_params: function get_layer_key: function hashables:
list measure_grobs: function merge: function override_elements: function params: list
process_layers: function setup_elements: function setup_params: function train: function
transform: function super: <ggproto object: Class GuideNone, Guide, gg> package_box:
function print: function process_layers: function setup: function subset_guides: function
train: function update_params: function super: <ggproto object: Class Guides, gg> @
mapping : Named list() @ theme : Named list() .. @ complete: logi FALSE .. @ validate:
logi TRUE @ coordinates:Classes 'CoordCartesian', 'Coord', 'ggproto', 'gg' <ggproto object:
Class CoordCartesian, Coord, gg> aspect: function backtransform_range: function clip: on
default: TRUE distance: function draw_panel: function expand: TRUE is_free: function

```

```

is_linear: function labels: function limits: list modify_scales: function range: function ratio:
NULL render_axis_h: function render_axis_v: function render_bg: function render_fg:
function reverse: none setup_data: function setup_layout: function setup_panel_guides:
function setup_panel_params: function setup_params: function train_panel_guides:
function transform: function super: <ggproto object: Class CoordCartesian, Coord, gg> @
facet :Classes 'FacetNull', 'Facet', 'ggproto', 'gg' <ggproto object: Class FacetNull, Facet,
gg> attach_axes: function attach_strips: function compute_layout: function draw_back:
function draw_front: function draw_labels: function draw_panel_content: function
draw_panels: function finish_data: function format_strip_labels: function init_gtable:
function init_scales: function map_data: function params: list set_panel_size: function
setup_data: function setup_panel_params: function setup_params: function shrink: TRUE
train_scales: function vars: function super: <ggproto object: Class FacetNull, Facet, gg>
@ layout :Classes 'Layout', 'ggproto', 'gg' <ggproto object: Class Layout, gg> coord: NULL
coord_params: list facet: NULL facet_params: list finish_data: function get_scales: func-
tion layout: NULL map_position: function panel_params: NULL panel_scales_x: NULL
panel_scales_y: NULL render: function render_labels: function reset_scales: function
resolve_label: function setup: function setup_panel_guides: function setup_panel_params:
function train_position: function super: <ggproto object: Class Layout, gg> @ labels :
Named list() @ meta : list() @ plot_env :<environment: R_GlobalEnv>

```

In this first chunk, we see that the data slot of the ggplot object now contains the underlying data.frame object. If we scroll past, we can spot additional slots, which are currently empty, waiting to be filled. We will sporadically check back and see how this changes. So, we have establish the first layer of our plot (Data). Now, we have to specify the next layer: aesthetics. In our case, we were interested in placing timepoint on the x-axis, and placing the TcellFrequency on the y-axis.

```

Plot <- ggplot(Data) + aes(x=timepoint, y=TcellFrequency)
Plot

```

(now we get a grey square, like before, and the x and y axis with white lines).

If we glanced behind the curtains, we now see the mapping slot has now been filled in:

```
str(Plot)
```

```

(I paste just the interesting fragment of the data obtained) @ mapping : List of 2 .. $ x:
language ~timepoint .. ..- attr(, "Environment")=<environment: R_GlobalEnv> .. $ y:
language ~TcellFrequency .. ..- attr(, "Environment")=<environment: R_GlobalEnv>

```

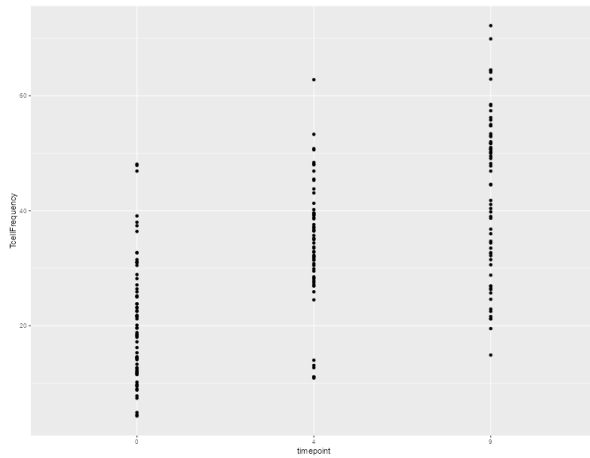
IMPORTANT NOTE: We should not put “ ” (quotation marks) around timepoint and Tcell-Frequency, because we use tidyverse to determine that they are columns and not objects. If we put quotation marks around them, we will obtain the two axis using those names as values (object) in the graphic.

Now we have to add a new layer: geometrics. We have to decide what geometries we want to use. This is the layer that determines how our data is displayed/plotted.

```
::: {.cell}
```

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) + geom_point()
```

```
:::
```

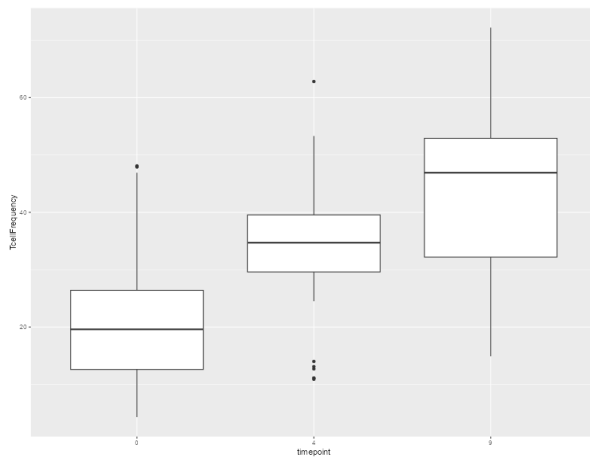


We could swap `geom_point()` out for `geom_boxplot()`:

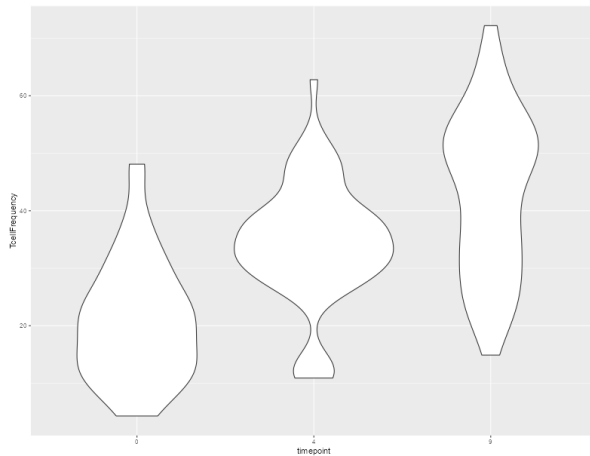
```
::: {.cell}
```

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) + geom_boxplot()
```

```
:::
```

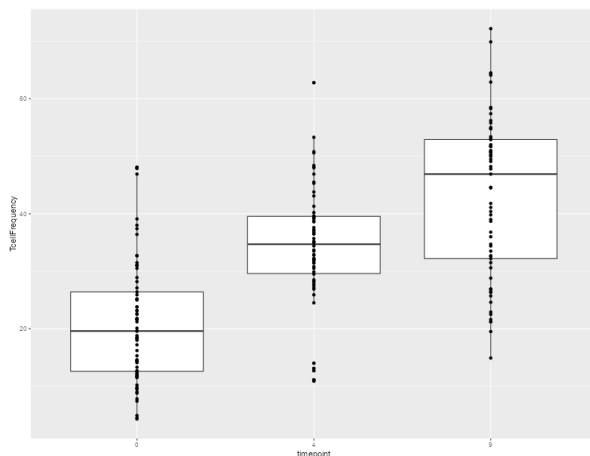


```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) + geom_violin()
```



We could also add two geometries

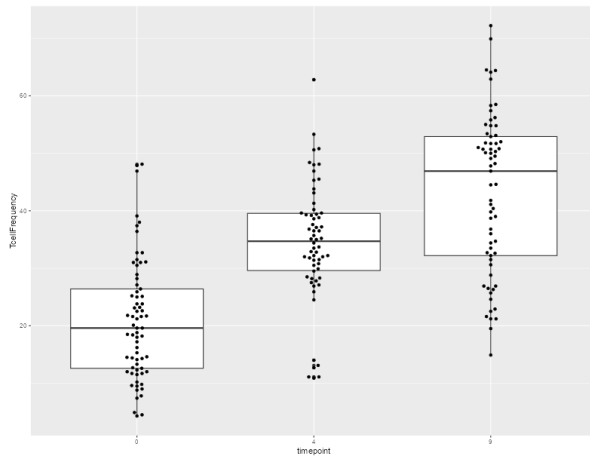
```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) + geom_boxplot() + geom_point()
```



As you can see, many different `geom_` functions exist in both `ggplot2`, as well as other R packages that expand the plotting capacity further. One I like to use for my own plots is the `ggbeeswarm` package, specifically the `geom_beeswarm()` function in conjunction with the `geom_boxplots()`.

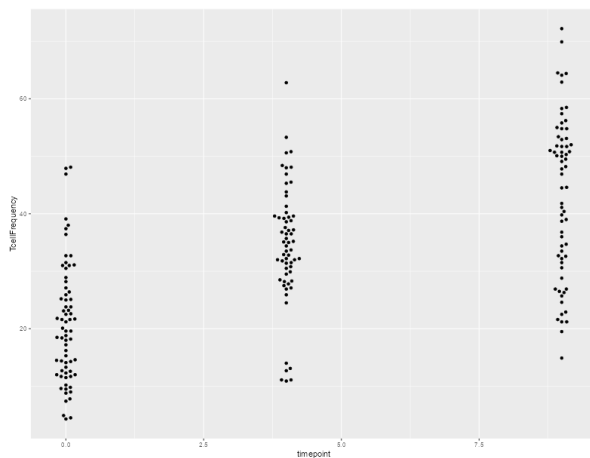
```
install.packages("ggbeeswarm")
```

```
library(ggbeeswarm)
Plot <- ggplot(Data) + aes(x=timepoint, y=TcellFrequency) + geom_boxplot() + geom_beeswarm()
Plot
```



In the case of the above dataset, we had converted timepoint to a factor. But what would our plot have looked like if we had left it in the original continuous format?

```
AlternateData <- read.csv(TheCSV, check.names=FALSE)
AlternateData <- AlternateData |>
  mutate(TcellProportion=Tcells_count/CD45_count) |>
  mutate(TcellFrequency=TcellProportion *100) |>
  mutate(TcellFrequency=round(TcellFrequency, 1))
ggplot(AlternateData) + aes(x=timepoint, y=TcellFrequency) + geom_beeswarm()
```



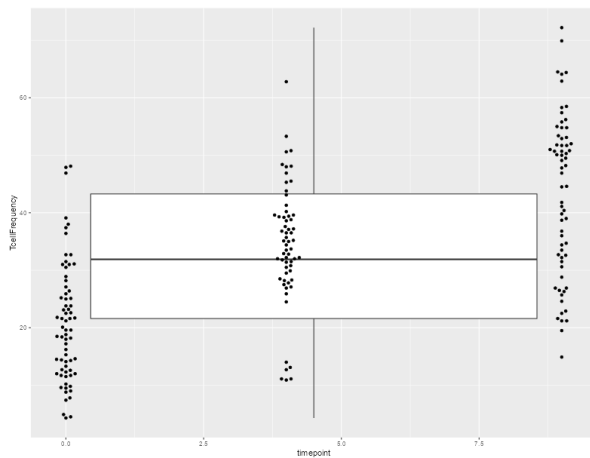
In this case, the x-axis instead of being spaced categorically, now appears as continuous, with the individual dots being plotted at 0, 4, 9 respectively. Given that `geom_boxplot` is expecting a categorical factor, this fails to plot correctly when added on:

```
::: {.cell}
```

```
ggplot(AlternateData) + aes(x=timepoint, y=TcellFrequency) + geom_boxplot() + geom_beeswarm()
```

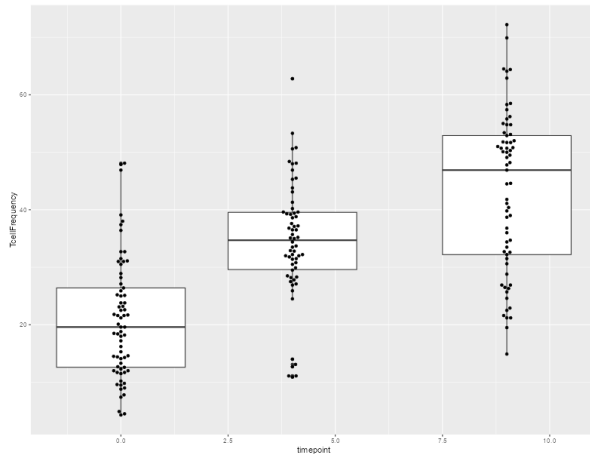
```
:::
```

Warning messages: 1: Orientation is not uniquely specified when both the x and y aesthetics are continuous. Picking default orientation 'x'. 2: Continuous x aesthetic did you forget `aes(group = ...)`?



Fortunately, the warning message provides some context we can use when troubleshooting. If we implement the suggestion, we are able to rescue `geom_boxplot()` plotting attempt even in absence of specifying `timepoint` as categorical.

```
ggplot(AlternateData) + aes(x=timepoint, y=TcellFrequency, group=timepoint) + geom_boxplot()
```



This example generally highlights the importance of being aware of what types of values you are trying to plot, warning messages, and some of the quirky odd-looking plots that can arise when the assumptions are not met.

Mismatched Assumptions

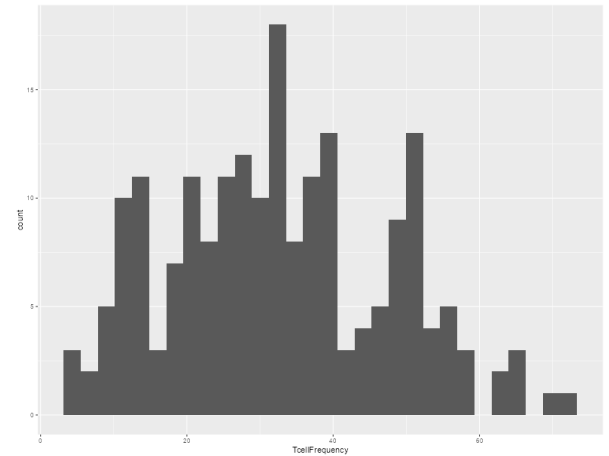
In this case, the two “geom_” functions we are using are using x-axis value as categorical, and y-axis as a continuous variable. What happens if we used a “geom_” function that has different specifications?

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) + geom_histogram()
```

Error in `geom_histogram()`: ! Problem while computing stat. Error occurred in the 1st layer. Caused by error in `setup_params()`: ! `stat_bin()` must only have an x or y aesthetic.

In the case of the above, `geom_histogram()` was expecting only one column being present in the `aes()` argument.

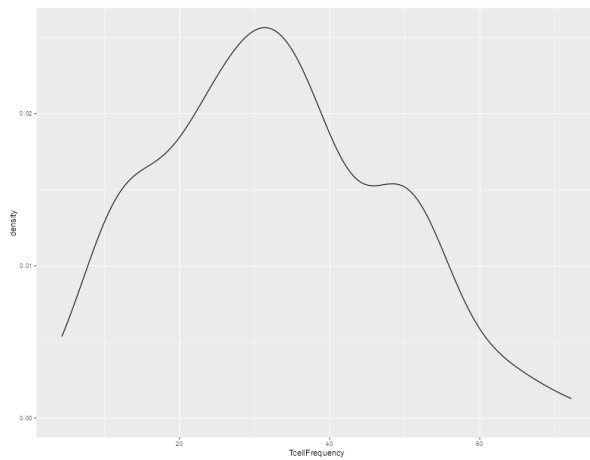
```
ggplot(Data) + aes(x=TcellFrequency) + geom_histogram()
```



`stat_bin()` using `bins = 30`. Pick better value `binwidth`.

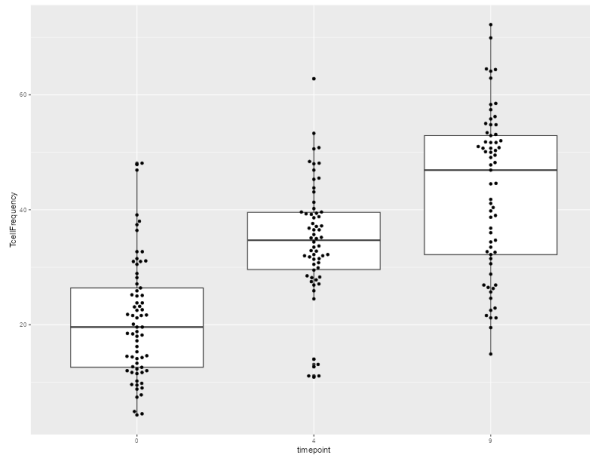
This would similarly be the case for `geom_density()`

```
ggplot(Data) + aes(x=TcellFrequency) + geom_density()
```



Let's back to our current beeswarm boxplot:

```
Plot <- ggplot(Data) + aes(x=timepoint, y=TcellFrequency) + geom_boxplot() + geom_beeswarm()
Plot
```



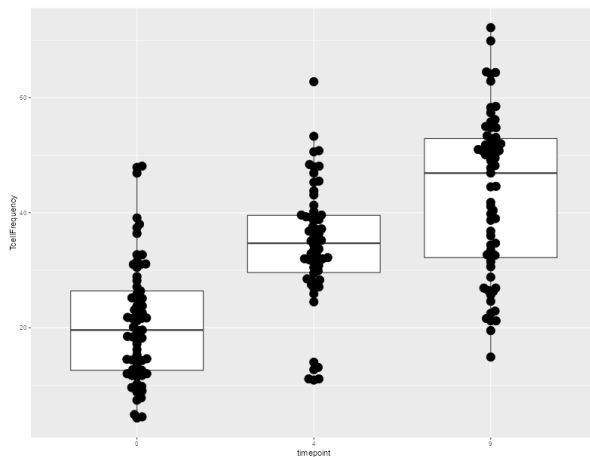
If we are making any large changes to the geometry layer, it would be occurring as an argument provided to the geometry determining functions. Lets check to see what our argument options are for both of the above.

```
?geom_boxplot
```

```
?geom_beeswarm
```

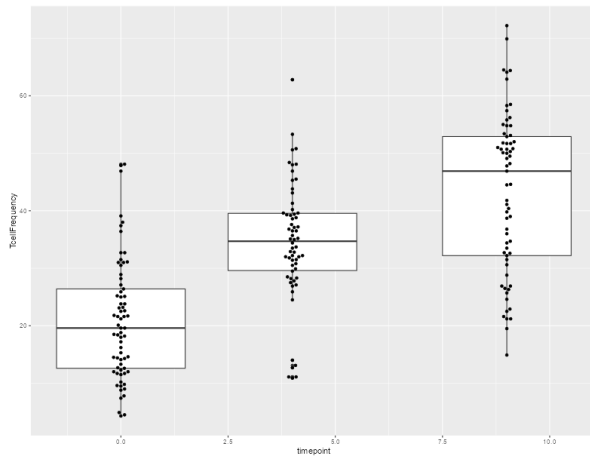
Currently, all the dots in the beeswarm are kind of small, and tightly wrapped together around the vertical axis for their respective groups. Let's attempt to change this. One of the easiest things to do when modifying arguments is to start with size (or the equivalent argument)

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +  
  geom_boxplot() + geom_beeswarm(size=5)
```



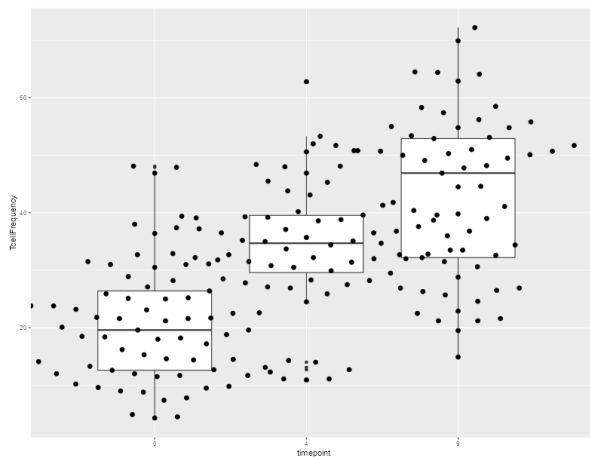
Let's tone it back.

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5)
```



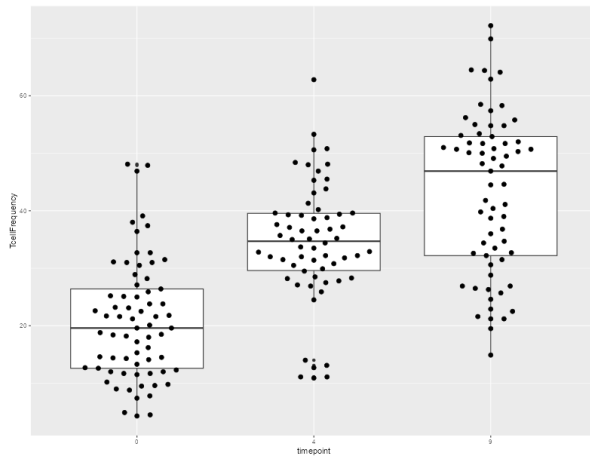
Better, but now the individual dots being too tightly wrapped together is limiting our capacity to visualize individuals. Lets next attempt to get the dots to spread out more horizontally from each other. Taking a look at the documentation, it looks like “cex” is the argument that might work.

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=5)
```



Too far! Let's adjust it again


```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +  
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5)
```



Good enough for now!

Scales

The next ggplot2 layer is `escales`. We have seen that we are able to change the size of dots and the space among them. However, we can also modify their appearance at the scale level. In our metadata, we can see that there is a column named “`infant_sex`”, that establish the sex origin of cells:

```
Data |> pull(infant_sex) |> unique()
```

```
[1] "Male" "Female"
```

Let’s go ahead and attempt to alter the dots to reflect based on this additional metadata variable. First, we can assign values based on the categorical values that are present for that column.

```
shape_sex <- c("Female" = 22, "Male" = 21)  
fill_sex <- c("Female" = "white", "Male" = "black")
```

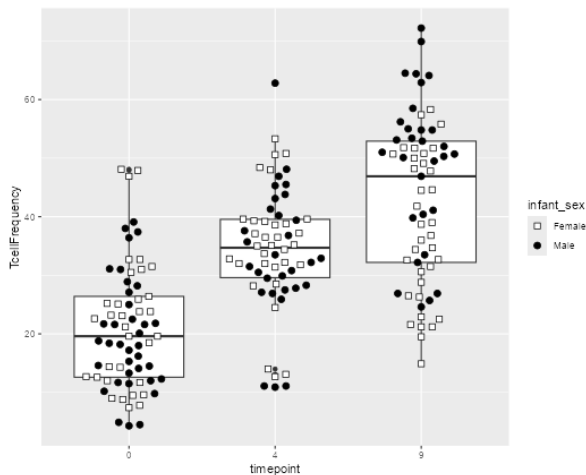
The two scale layer functions we will use will be `scale_shape_manual()` and `scale_fill_manual()`.

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5) +
  scale_shape_manual(shape=shape_sex) + scale_fill_manual(fill=fill_sex)
```

Error in discrete_scale(): ! unused argument (shape = c(22, 21))

This did not quite work as expected, with our additional arguments not being perceived by the plot. Part of this is due to needing to set up the underlying geometry layer with the necessary arguments as well. After some investigation, let's try

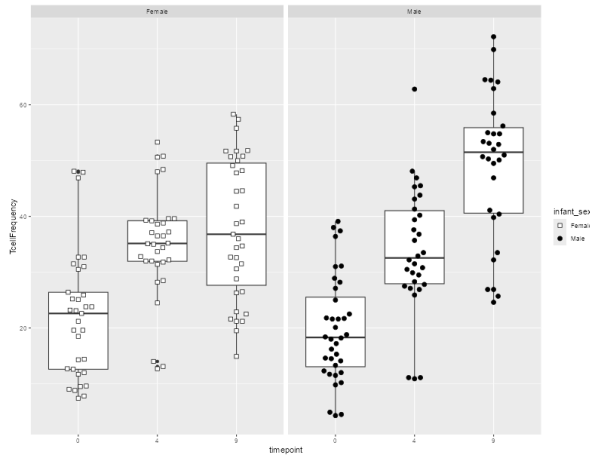
```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex)
```



Facets

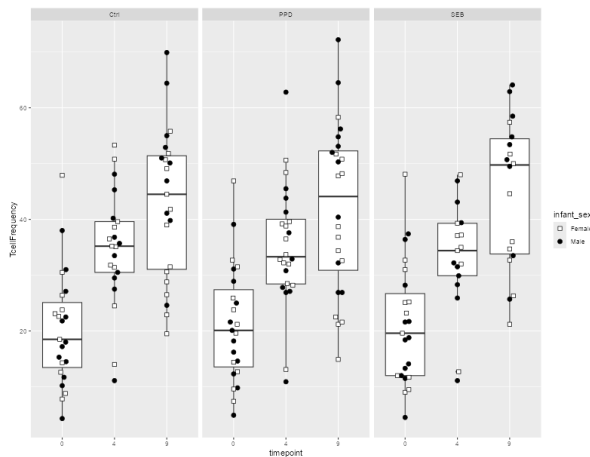
At this point, we have added the Data, Aesthetic, and two Geometry layers. The next layer, Facets, allows creation of separate plots based on an additional variable. For example, if we were interested in seeing the difference between male and female infants, we could specify the column name within the `facet_wrap()` argument.

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex) +
  facet_wrap(~ infant_sex)
```



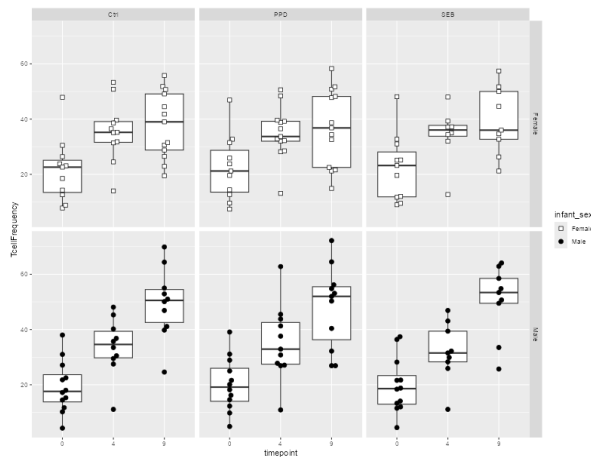
We could similarly return individual plots for various treatment conditions (Ctrl, PPD, SEB)

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex) +
  facet_wrap(~ Condition)
```



We could also use the variant `facet_grid()` to facet on the basis of two separate variables.

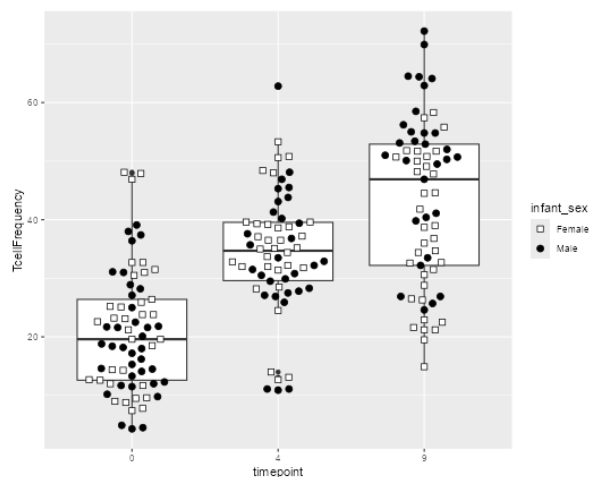
```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex) +
  facet_grid(infant_sex ~ Condition)
```



Coordinates

Having built out much of the underlying plot, we currently have the following plot

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex)
```



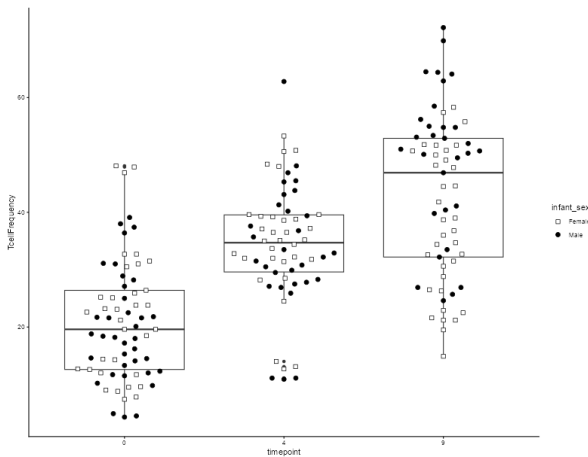
Themes

The final layer in today's exploration of ggplot2 is the "theme" layer. This is where most of the customizations tend to occur when it comes to editing our plots, as most of these are

often personal choices. There are a couple ways by which we can implement these, either using a broader “theme_” function, which will set to various defaults, or through the general theme() function, which as long as we look up the correct argument will let us edit pretty much anything.

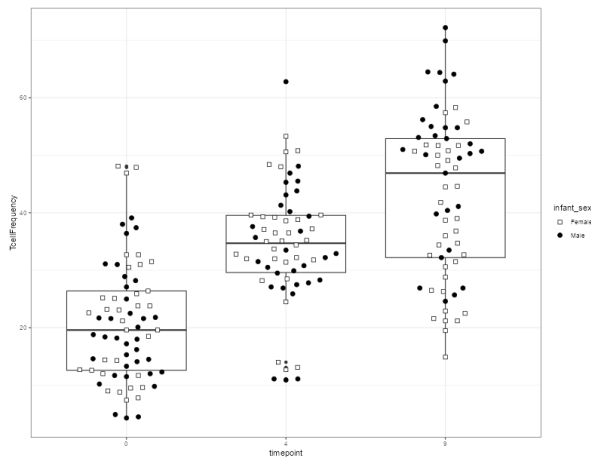
Here are a couple examples of what applying one of the various default “theme_”’s looks like.

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex) +
  theme_classic()
```



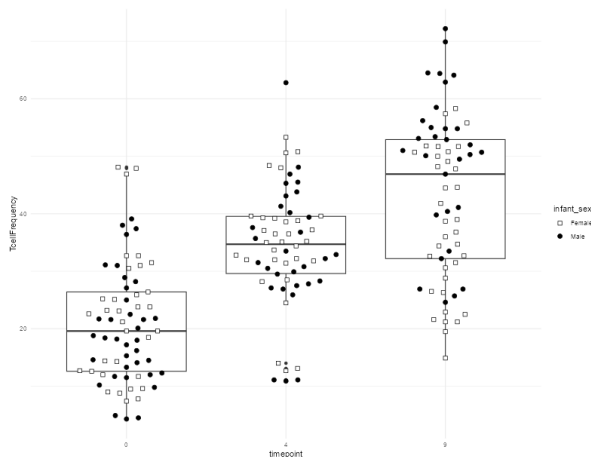
(in this case, background is white and there is no lines from axis)

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex) + theme_bw()
```



(here, the background is white and appears lines from axis. Also there is an square surrounding the plot)

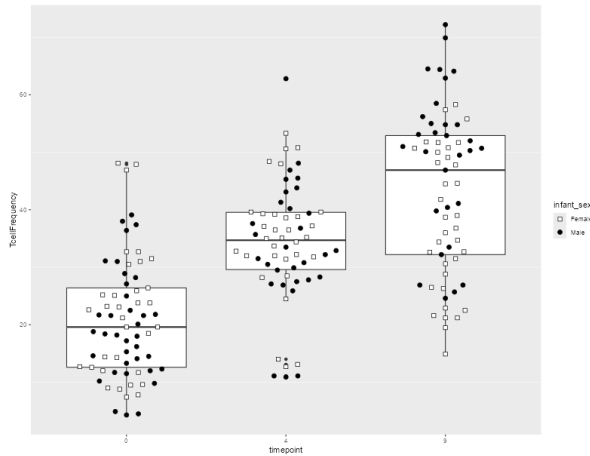
```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex) +
  theme_minimal()
```



(background in white, no axis or square)

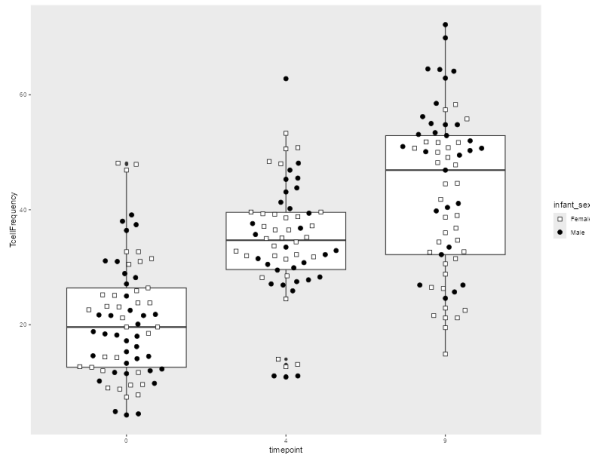
By contrast, instead of (or in addition to) applying a broad theme, we can also modify individual elements of our plots. For example, within theme functions parenthesis, we can silence a particular plot element by setting its theme element equal to the element_blank(). If we were trying to remove the major grid lines, we would run a line of code like this:

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  + scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex) +
  theme(
    panel.grid.major = element_blank()
```



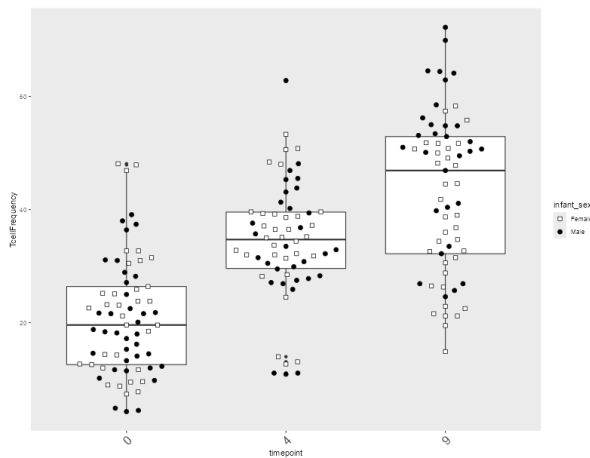
If we decided to remove the minor grid lines, we would set it's equivalent argument equal to `element_blank()` within the `theme()`, although we must first add a comma to distinguish it from the previous argument.

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  + scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex)
  + theme(
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank())
```



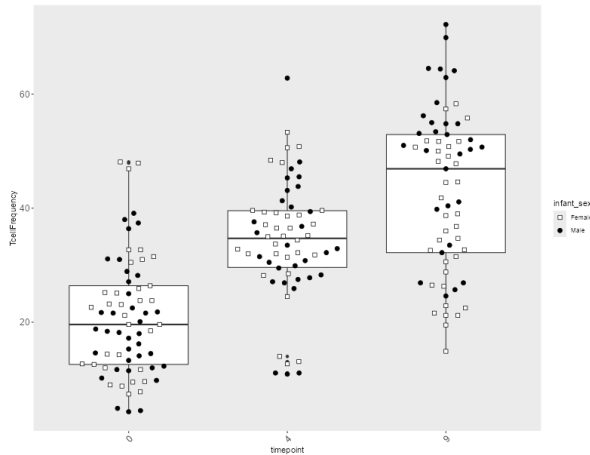
In addition to removing things, we can also modify theme elements. One issue often encountered is the axis-text labels obscure one another. One solution is to rotate them (via the “angle” argument) and horizontally justify their starting position using “hjust” to line up with the original starting position.

```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex)) +
  scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex) +
  theme(
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.text.x = element_text(angle=45, hjust=1, size = 16))
```



We can in a similar manner adjust the size of the axis text labels, using the “size” argument.


```
ggplot(Data) + aes(x=timepoint, y=TcellFrequency) +
  geom_boxplot() + geom_beeswarm(size=2.5, cex=2.5, aes(shape=infant_sex, fill=infant_sex))
+ scale_shape_manual(values=shape_sex) + scale_fill_manual(values=fill_sex)
+ theme(
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  axis.text.x = element_text(angle=45, hjust=1, size = 12),
  axis.text.y = element_text(size=12))
```



ggcyto

Let's begin by loading in the additional R packages we will need

```
library(CytoML)
library(ggcyto)
```

Loading required package: flowCore Loading required package: ncdfFlow Loading required package: BH Loading required package: flowWorkspace As part of improvements to flowWorkspace, some behavior of GatingSet objects has changed. For details, please read the section titled “The cytoframe and cytoset classes” in the package vignette:

vignette(“flowWorkspace-Introduction”, “flowWorkspace”)

We are going to use again the example from week5 to obtain a GatingSet object:

```
::: {.cell}
```

```
Folder <- file.path("C:/Users/Macarena/CytometryInR/course/05_GatingSets/data")
Folder
```

...

```
FlowJoWsp <- list.files(path = Folder, pattern=".wsp", full = TRUE)
FlowJoWsp
```

```
ThisWorkspace <- FlowJoWsp[stringr::str_detect(FlowJoWsp, "Opened")]
ThisWorkspace
```

```
ws <- open_flowjo_xml(ThisWorkspace)
ws
```

```
gs <- flowjo_to_gatingset(ws=ws, name=1, path=Folder, additional.keys = "GROUPNAME")
gs
```

And return to the plot we had left off on:

```
Plot <- ggcyto(gs[6], subset="Tcells", aes(x="CD8", y="CD4")) + geom_hex(bins=100)
```

And we are going to use some ggplot2 functions:

```
Plot + theme_bw()
```

